

Running BOTUX with Docker Guide

This guide walks you through the steps required to install Docker and run the entire BOTUX system (including the PostgreSQL database, backend API, and frontend dashboard) using Docker.

1. Install Docker & Docker Compose

To run this project, you need to install **Docker Desktop** (for macOS and Windows) or **Docker Engine** (for Linux). Docker Desktop includes the `docker compose` CLI tool by default.

On macOS

- **Method 1 (Direct Download):** Visit the official [Docker Desktop for Mac](#) page and download the installer matching your processor chip (Apple Silicon or Intel).
- **Method 2 (Using Homebrew):** Run the following command in your Terminal:

```
brew install --cask docker
```

- After installation, open the Docker application from your Launchpad to start the Docker daemon.

On Windows

- Visit [Docker Desktop for Windows](#) to download the installer.
- During installation, make sure to enable the **WSL 2 backend** (Windows Subsystem for Linux) to optimize performance.
- Restart your computer if prompted, then launch Docker Desktop.

On Linux (Ubuntu/Debian)

Run the following commands in your Terminal to install Docker Engine and the Docker Compose plugin:

```
# Update package index
sudo apt-get update

# Install packages to allow apt to use a repository over HTTPS
sudo apt-get install ca-certificates curl gnupg

# Add Docker's official GPG key
sudo install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o
/etc/apt/keyrings/docker.gpg
sudo chmod a+r /etc/apt/keyrings/docker.gpg

# Set up the repository
echo \
  "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg]
https://download.docker.com/linux/ubuntu \
  $(. /etc/os-release && echo "$VERSION_CODENAME") stable" | \
  sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

# Install Docker and Docker Compose
sudo apt-get update
sudo apt-get install docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin

# Enable and start the Docker service
sudo systemctl enable docker
```

```
sudo systemctl start docker
```

2. Prepare the Environment File (.env)

Before starting Docker, create an .env environment file from the .env.example template:

1. Navigate to the project directory:

```
cd botux
```

2. Copy .env.example to create .env:

```
cp .env.example .env
```

Note: Inside the Docker environment, the API container automatically connects to the database using the address of the postgres service within the Docker network (postgres://botux:botux@postgres:5432/botux). You do not need to modify the local BOTUX_DB_URI setting in your .env file because Docker Compose overrides this variable dynamically at runtime.

3. Launch the System Using Docker Compose

The docker-compose.yml file in the project defines 3 services running together in an isolated network:

- **postgres:** PostgreSQL 16 database (binds to port 5432 on your host machine).
- **api:** FastAPI backend (binds to port 8001 on your host machine).
- **dashboard:** Next.js frontend (binds to port 3001 on your host machine).

To start the system:

```
docker compose up --build -d
```

- **--build:** Rebuilds Docker images for the API and Dashboard to ensure they run with the latest source code changes.
- **-d:** Runs the containers in the background (detached mode).

To check container status:

```
docker compose ps
```

All three containers (botux-postgres, botux-api, and botux-dashboard) should show a status of Up or running.

4. Accessing Services

Once the containers are successfully running:

- **Dashboard:** <http://localhost:3001>
- **API Swagger UI (Documentation):** <http://localhost:8001/docs>
- **API Health Check:** <http://localhost:8001/health>

5. Management Commands

Viewing container logs:

```
# View real-time logs for all services
docker compose logs -f
```

```
# View real-time logs for the API service only
docker compose logs -f api
```

Stopping the system:

```
docker compose down
```

Cleaning and resetting the database:

If you want to completely clear and reset the database:

```
docker compose down -v
```

(The `-v`` flag deletes the database volume named `botux_pgdata``. When you run `docker compose up`` again, it will initialize a completely clean, empty database).